

UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

ATIVIDADES PRÁTICAS SUPERVISIONADAS

- APS

**AS TÉCNICAS CRIPTOGRÁFICAS, CONCEITOS, USOS E
APLICAÇÕES**

Joaquim José Mendes de Souza - N079657

João Vitor Oliveira de Vasconcellos - G7733J3

Leandro Martins de Souza - G853JJ8

Lucas Inocência de Franca - N239239

Nicolas Pires de Souza - G7745G0

Rodrigo Pamplona Soares - N2409H8

Washington dos Santos Souza Filho- N046FD9

São José dos Campos

2023

UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

ATIVIDADES PRÁTICAS SUPERVISIONADAS
- APS

AS TÉCNICAS CRIPTOGRÁFICAS, CONCEITOS, USOS E
APLICAÇÕES

Atividades Práticas

Supervisionadas do 1º e 2º Semestres
do Curso de Ciência da Computação
da **Universidade Paulista – UNIP**.

Coordenador: Prof. Fernando A.

Gotti

Profs Responsável: André

Yoshimi **Kusumoto**

Sumário

1 INTRODUÇÃO	4
2 CRIPTOGRAFIA (CONCEITOS GERAIS).....	5
3 TÉCNICAS CRIPTOGRÁFICAS MAIS UTILIZADAS	7
4 DISSERTAÇÃO	9
Protocolos de segurança:	13
Criptografia de Disco:	13
Ataques por Texto Claro Conhecido (Known-Plaintext Attacks):	13
Falha em Casos de Ataques a Blocos de Dados:.....	13
Ataques de Força Bruta:	13
Análise de Frequência:	13
Chaves curtas:.....	14
Melhorias:	14
5 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO.....	16
6 RELATÓRIOS COM LINHAS DE CÓDIGO	18
REFERENCIAS.....	23

1 INTRODUÇÃO

Nesse trabalho será abordado sobre as técnicas de criptografia, conceitos, usos e suas aplicações. Como objetivo principal desse trabalho, criamos um código, para criptografar e descriptografar frases, textos entre outros.

O trabalho está organizado em seis partes. na primeira parte falaremos sobre os conceitos gerais da criptografia, onde aborda métodos, definições, entre outros. Na segunda parte, iremos apresentar os tipos de técnicas que são mais utilizadas, e como elas funcionam, se é através de chaves, senhas ou códigos. na terceira parte, falaremos do método e da técnica XOR, método esse que escolhemos para a realização do nosso código, e também, será abordado suas vantagens e desvantagens que existem ao utiliza-lo Na quarta colocaremos nosso plano de desenvolvimento, e algumas das ferramentas usadas para criação do código. na quinta parte, iremos mostrar nossas linhas de códigos, nelas há os comentários para entendermos melhor o que cada função faz e por fim, na sexta parte, as referências.

2 CRIPTOGRAFIA (CONCEITOS GERAIS)

Criptografia consiste na prática de proteção da informação por meio do uso de codificadores, hashes, entre outros. As informações podem estar como um arquivo em um disco, em trânsito ou em uso com a computação de dados.

Existe 3 objetivos principais:

- Confidencialidade
- Integridade
- Autenticação

Para que serve a criptografia?

A criptografia tem origem nas informações confidenciais entre figuras militares e políticas as mensagens tinham que ser secretas e parecer aleatórias para confundir pessoas que interceptassem e não tivessem a chave para descriptografia.

Hoje em dia com o avanço da segurança, os algoritmos dependem de análises e matemática para sua segurança. Com a evolução da segurança também evoluiu a quantidade de métodos de segurança, como autenticação de mensagens tanto de texto quanto de áudio, proteção de dados, entre outros.

A criptografia simétrica exige que todos os destinatários da mensagem tenham acesso a uma chave compartilhada. Os algoritmos modernos usam matemática avançada e uma ou mais chaves de criptografia. Isto torna relativamente fácil criptografar uma mensagem, mas praticamente impossível descriptografá-la sem conhecer as chaves. As técnicas de criptografia se enquadram em duas categorias, simétricas e assimétricas, com base no funcionamento de suas chaves.

A criptografia assimétrica (ou de chave pública) consiste em um amplo conjunto de algoritmos. Estes são baseados em problemas matemáticos que são relativamente fáceis de executar em uma direção, mas que não podem ser facilmente revertidos. Uma das cifras de bloco mais populares é o Advanced Encryption Standard (AES)

Um esquema de criptografia é chamado de assimétrico se usa uma chave (a chave pública) para criptografar dados e uma chave diferente, mas matematicamente relacionada, para descriptografar os dados. Deve ser computacionalmente inviável determinar a chave privada se a única coisa conhecida for a chave pública. Como algoritmos de chave pública como RSA-OAEP são menos eficientes que seus equivalentes simétricos, eles não são comumente usados para criptografar dados.

A criptografia de chave pública é usada para resolver trocas de chaves usando criptografia simétrica. Diffie-Hellman é um tipo diferente de algoritmo criptográfico de chave pública. Ele foi projetado para ajudar as partes a chegarem a um acordo sobre uma chave simétrica na ausência de um canal seguro. A criptografia híbrida é amplamente utilizada em protocolos de transferência de dados para a Web, como HTTPS.

3 TÉCNICAS CRIPTOGRÁFICAS MAIS UTILIZADAS

RSA:

Algoritmo de criptografia assimétrica que é usado para criptografia de chave pública. Recomendado para autenticação e troca segura de chaves.

ECC (Elliptic Curve Cryptography):

Criptografia assimétrica que é conhecida por eficiência e segurança. Útil em apps com recursos limitados, como celulares e sistemas embarcados.

Criptografia hash:

Cria um valor fixo (hash) de tamanho fixo a partir do conjunto de dados. Quaisquer alterações nos dados resultarão em um resumo completamente diferente.

Importância: Usado para verificar a integridade dos dados e criar assinaturas digitais.

Criptografia assimétrica (ou de chave pública):

Como funciona: usa chaves públicas para criptografar dados e chaves privadas para descriptografá-los. A chave pública pode ser compartilhada livremente, enquanto a chave privada deve ser mantida em segredo.

Resolve o problema de compartilhamento de chaves na criptografia simétrica. É amplamente utilizado para autenticação, assinaturas digitais e segurança de comunicações online.

Blowfish:

Conhecido por sua velocidade e eficiência e usado para segurança de dados, é um algoritmo de chave simétrica e também considerado menos seguro que o AES.

Criptografia ponta a ponta:

Ação: Garante que apenas remetentes e destinatários autorizados possam ler mensagens. As mensagens são criptografadas no dispositivo do remetente e descriptografadas no dispositivo do destinatário. Importância: Garante a segurança evitando que terceiros, inclusive provedores, acessem as mensagens.

AES (Advanced Encryption Standard):

AES é considerado um dos mais seguros. Ele opera com chaves de tamanho fixo (128, 192 ou 256 bits) usados em vários apps, incluindo comunicações seguras e criptografia de dados em repouso.

DES (Data Encryption Standard):

Foi um dos primeiros algoritmos para segurança de dados. Foi substituído pelo AES devido a preocupações com a segurança.

4 DISSERTAÇÃO

A operação XOR (ou exclusivo OU) é uma operação lógica que compara dois bits. Ela retorna verdadeiro (ou 1) se exatamente um dos bits de entrada for 1, e falso (ou 0) se ambos os bits de entrada forem iguais (ambos 0 ou ambos 1). A operação XOR é frequentemente denotada pelo símbolo $\oplus$ ou $\wedge$. Aqui está a tabela verdade para a operação XOR:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Estruturação:

A operação XOR é fundamentalmente uma operação bitwise, o que significa que ela é aplicada individualmente a cada par de bits correspondentes em dois números binários. Por exemplo, se você tiver dois números binários $A = 1010$ e $B = 1100$, o resultado da operação XOR seria 0110.

Fundamentos e Conceitos:

1. Propriedade de Reversibilidade:

A operação XOR é reversível, o que significa que se você aplicar XOR novamente ao resultado usando o mesmo valor com o qual você fez a operação XOR inicial, você obterá de volta o valor original. Isso é fundamental para a criptografia, onde a mesma chave é usada para cifrar e decifrar dados.

2. Aplicações em Criptografia:

XOR é amplamente usado em algoritmos criptográficos, especialmente em cifras de fluxo. Em cifras de fluxo, a chave é combinada com os dados usando XOR para produzir o texto cifrado. Uma das cifras de fluxo mais conhecidas que usa XOR é o algoritmo RC4.

3. Operações Bitwise:

XOR é uma operação bitwise, o que significa que ela opera em cada par de bits de entrada independentemente. Isso é útil em muitas aplicações onde você precisa manipular dados em um nível de bits.

4. Propriedade de Odd Parity:

XOR é usado para verificar a paridade dos bits. Quando você aplica XOR a um conjunto de bits e obtém 1 como resultado, significa que há um número ímpar de bits 1 no conjunto.

5. Máscaras de Bits e Manipulação de Dados:

XOR é usado para aplicar máscaras de bits em dados. Ao fazer XOR com uma máscara de bits, você pode alterar seletivamente certos bits em um valor sem afetar outros bits.

6. Geração de Sequência Aleatória:

XOR pode ser usado em algoritmos para gerar sequências pseudoaleatórias, que são úteis em diversas aplicações, incluindo criptografia.

Esses são alguns dos fundamentos e conceitos associados à operação XOR. Sua simplicidade e versatilidade tornam-na uma ferramenta valiosa em muitos campos, especialmente na criptografia e na manipulação eficiente de dados em nível de bits.

O método XOR (ou exclusivo OU) não é um método de criptografia por si só, mas é uma operação lógica fundamental que é amplamente utilizada em algoritmos de criptografia para diferentes propósitos. No contexto da

criptografia, a operação XOR é usada para combinar dados de maneira que seja difícil ou impossível reverter o processo sem a chave correta. Embora não seja um método novo em relação a métodos antigos de criptografia, a operação XOR oferece algumas características únicas que a tornam valiosa em esquemas de criptografia:

1. Simplicidade: A operação XOR é fácil de entender e implementar, o que a torna atraente para algoritmos de criptografia simples e eficientes.

2. Reversibilidade: A operação XOR é reversível, o que significa que, se você aplicar a operação novamente aos dados criptografados com a mesma chave, você obterá os dados originais. Isso é útil para descriptografar os dados de forma eficaz.

3. Propriedades matemáticas: A operação XOR possui propriedades matemáticas úteis. Por exemplo, quando um mesmo valor é XOR-ed com uma chave duas vezes, o resultado é o valor original, o que é uma propriedade importante em muitos algoritmos criptográficos.

4. Aplicações em algoritmos de fluxo: A operação XOR é comumente usada em cifras de fluxo, onde bits da chave são combinados com os bits dos dados para produzir uma sequência cifrada. O algoritmo de cifra de fluxo RC4, por exemplo, usa a operação XOR para combinar a chave com os dados.

5. Máscaras e criptografia de um só sentido: XOR também é usado em técnicas como máscaras (máscaras de bits) e funções de hash criptográficas para criar valores de hash ou para obscurecer informações, tornando difícil para os invasores recuperarem dados originais sem a chave apropriada.

6. Operações bitwise: XOR é uma operação bitwise que opera em nível de bits. Isso permite manipular dados em um nível granular, o que é útil em muitos algoritmos criptográficos.

Em resumo, enquanto a operação XOR em si não trouxe algo totalmente novo em relação a métodos antigos de criptografia, suas propriedades e simplicidade a tornam uma ferramenta valiosa e amplamente utilizada em muitos algoritmos de criptografia modernos.

‘O XOR é mais frequentemente utilizado em programação e computação de baixo nível’

Criptografia XOR

É uma técnica simples, para criptografar e descriptografar dados. Se baseia na operação lógica XOR, que é usada para combinar bits de dados.

Operação XOR

Operação XOR, aceita dois valores binários e retorna um resultado 1 se um dos bits for realmente 1, e 0 caso contrário. Segue a tabela XOR:

$$0 \text{ XOR } 0 = 0$$

$$0 \text{ XOR } 1 = 1$$

$$1 \text{ XOR } 0 = 1$$

$$1 \text{ XOR } 1 = 0$$

XOR

É um algoritmo de criptografia simétrica, que significa que a mesma linguagem é usada para criptografar dados como descriptografar. É rápido e eficiente, usado corretamente. Por isso é frequentemente usada em aplicativos de baixo nível e protocolos de rede.

É vulnerável a ataques de força bruta se o espaço das chaves for pequeno ou se a chave for usada mais de uma vez.

É usado em aplicações como mascaramento de bits.

Limitações

A maior limitação do XOR é que ele não fornece uma segurança muito boa comparada com os outros algoritmos, como AES (Padrão de Criptografia Avançada). XOR não é recomendado para aplicações de segurança de alto nível, ataques de força bruta pode ser sua fraqueza.

Protocolos de segurança:

Como funciona? Envolve a combinação de tecnologias criptográficas com determinados protocolos para proteger as comunicações de rede, como SSL/TLS para segurança de rede. Importância: Garante a confidencialidade, autenticidade e integridade dos dados transmitidos pela Internet.

Criptografia de Disco:

Funcionamento: Criptografa todo o conteúdo de um disco ou volume de armazenamento, protegendo-o contra acesso não autorizado.

Importância: Protege dados armazenados em dispositivos, impedindo que sejam lidos por pessoas não autorizadas no caso de perda ou roubo do dispositivo.

- Possíveis bugs e pontos fracos podem ser encontrados no XOR:

Ataques por Texto Claro Conhecido (Known-Plaintext Attacks):

Se o hacker tiver acesso a versão cifrada e também souber o conteúdo original, ele pode usar essa informação para deduzir e tentar descobrir a chave de criptografia, e decifrar outros dados com a mesma chave.

Falha em Casos de Ataques a Blocos de Dados:

É frequentemente usado em apps que não quebram os dados em bloco, que pode torná-lo vulnerável a ataques que exploram padrões em blocos de dados criptografados.

Ataques de Força Bruta:

XOR é vulnerável a ataques de força bruta. o atacante pode tentar todas as combinações possíveis de chaves até encontrar a chave correta e recuperar os dados originais.

Análise de Frequência:

Mesmo que os dados estejam criptografados, é possível realizar uma análise e frequência criptografados para inferir informações sobre o conteúdo original.

Chaves curtas:

O XOR é dependente do tamanho e da complexidade da chave. Chaves curtas e previsíveis podem tornar a criptografia facilmente quebrável.

É importante lembrar que meios que precisam de mais segurança de dados é possível usar algoritmos de criptografia, como AES, RSA, ECC, eles garantem mais segurança.

Melhorias:**Autenticidade de Chaves:**

Podemos dizer que uma das melhorias do algoritmo seria o uso de geradores de números aleatórios, para que não tenha chaves repetidas e previsíveis.

Combinação com Outras Técnicas de Criptografia:

Pode ser combinado com outras técnicas de criptografia, para fornecer camadas adicionais de segurança. Para obter sistemas mais fortes e seguros.

Uso em Algoritmos de Cifra de Fluxo:

Algoritmos de cifra de fluxo, podem ser aprimorados para obter uma melhor segurança. Pode envolver o desenvolvimento de novos algoritmos de cifra de fluxo ou a otimização de dados.

As consequências dessas melhorias podem incluir:**Maior Segurança:**

Uma das consequências leva a uma maior segurança. Podemos deixar os sistemas mais seguros, menos vulneráveis a ataques e violações.

Maior proteção de dados:

Melhorias na criptografia XOR podem levar a uma maior proteção de dados sensíveis em aplicações que dependem dessa operação.

Melhor Resistência a Ataques Criptoanalíticos:

Melhorias na geração de chaves, podem aumentar a resistência a ataques e garantir que os dados permaneçam seguros.

A segurança na criptografia é um campo em constante evolução, está sempre evoluindo para lidar com vários tipos de ameaças.

5 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Na aplicação, é utilizado o método XOR para a criptografia, que utiliza a operação binária na qual, se caso os bits da chave e do texto forem idênticos, o resultado será 0, caso sejam diferentes, o resultado será 1.

O programa foi desenvolvido com sete funções destinadas a criptografar um texto e, em seguida, descriptografá-lo, utilizando a mesma chave fornecida pelo usuário.

Para garantir a eficiência, navegabilidade e eficácia do software, foi adicionado um menu com 5 opções, escolhendo o número de cada opção do menu, você entrará para a função selecionada. Foi usado um loop "while True" que só é parado escolhendo a opção de número 5, onde há um "break" que irá encerrar a execução do programa.

A primeira função é designada para a criptografia, a qual recebe um texto e uma chave. Em seguida, uma lista é inicializada para armazenar os bits resultantes do processo binário, onde um loop "for" é empregado para percorrer o texto e a chave, comparando-os e utilizando o operador "^", que representa a operação binária XOR.

A segunda função do código é dedicada à criação de uma chave aleatória. Para realizar essa tarefa, são importadas as bibliotecas random e string, que auxiliam na geração da chave. A biblioteca random é usada para acessar a biblioteca string, permitindo a seleção aleatória de caracteres e dígitos para gerar uma chave de 32 caracteres. Logo em seguida, usamos a quarta função para armazená-la em um arquivo de texto ".txt" que falaremos mais sobre abaixo.

A terceira função verifica se a chave está no dicionário e foi criada, caso contrário, uma mensagem informa que a chave não foi encontrada. Ainda na terceira função, ela tem como propósito verificar se a chave está

presente no dicionário e foi criada. Caso não seja encontrada uma chave, será exibida uma mensagem indicando que não foi encontrada.

Já a quarta função é responsável por salvar as chaves em um arquivo no formato ".txt" que, no início do código, é declarado o dicionário chamado "chaves". Em seguida, o programa abre o arquivo no modo de leitura para consultar as chaves que estão registradas. Este processo permite armazenar e recuperar as chaves, proporcionando persistência aos dados utilizados na aplicação.

A quinta e sexta função têm como único propósito verificar se a frase inserida pelo usuário é hexadecimal válidos. Para realizar essa verificação, o conteúdo inserido pelo usuário é processado na sexta função, que nesse caso, é empregado o bloco "try" para tentar converter a string de entrada em formato hexadecimal utilizando o método fromhex(). Caso a conversão não seja bem-sucedida, uma mensagem é exibida ao usuário informando que a string não era um valor hexadecimal válido. Esta abordagem garante que apenas entradas válidas em hexadecimal sejam aceitas pela aplicação, melhorando a segurança evitando possíveis erros e consistência.

A sétima função é responsável pela etapa de descriptografia da frase ou texto inserido pelo usuário. Ela recebe o texto criptografado, após passar pela função de criptografia. Nesta etapa, realiza-se a operação reversa, comparando bit a bit, utilizando a chave inicialmente empregada na função de criptografia para fazer a "engenharia reversa".

6 RELATÓRIOS COM LINHAS DE CÓDIGO

```
import random
import string

# Declarando classes
class Criptografia:
    def __init__(self):
        self.chaves = {}
        self.carregar_chaves()

# Função para criptografar
def criptografar(self, texto, chave):
    criptografado = []
    for i in range(len(texto)):
        criptografado.append(texto[i] ^ chave[i % len(chave)])
    return bytes(criptografado)

# Função para gerar uma chave aleatoria e salvar
def criar_chave_aleatoria(self, nome_chave):
    random_key = "".join(random.choice(string.ascii_letters + string.digits) for
_ in range(32))
    self.chaves[nome_chave] = random_key.encode('utf-8')
    self.salvar_chaves()

# Função para buscar a chave e mostrar a 'string' associada a ela
def buscar_chave(self, nome_chave):
    if nome_chave in self.chaves:
        return self.chaves[nome_chave]
    try:
        return bytes.fromhex(nome_chave)
    except ValueError:
        return None
```

```

# Função para salvar as chaves em um arquivo '.txt'
def salvar_chaves(self):
    with open("chaves.txt", "w") as arquivo:
        for chave_nome, chave in self.chaves.items():
            arquivo.write(f"{chave_nome}:{chave.hex()}\n")

# Função para carregar as chaves ja salvas
def carregar_chaves(self):
    try:
        with open("chaves.txt", "r") as arquivo:
            linhas = arquivo.readlines()
            for linha in linhas:
                partes = linha.strip().split(":")
                if len(partes) == 2:
                    chave_nome, chave = partes
                    self.chaves[chave_nome] = bytes.fromhex(chave)
    except FileNotFoundError:
        pass

# Função para verificar se a mensagem criptografada foi inserida em
Hexadecimal
def input_mensagem_cifrada_hex(self):
    while True:
        mensagem_cifrada_hex = input("Insira a frase criptografada em
hexadecimal: ")
        if self.is_hexadecimal(mensagem_cifrada_hex):
            return mensagem_cifrada_hex
        else:
            print("O texto inserido não é um valor hexadecimal válido. Tente
novamente.")

def is_hexadecimal(self, texto):

```

```

try:
    bytes.fromhex(texto)
    return True
except ValueError:
    return False

# Função para descriptografar
def descriptografar(self, texto_criptografado, chave):
    descriptografado = []
    for i in range(len(texto_criptografado)):
        descriptografado.append(texto_criptografado[i] ^ chave[i %
len(chave)])
    return bytes(descriptografado)

# Aqui inicia o programa
if __name__ == "__main__":
    criptografia = Criptografia()

# Menu do programa com as opções disponíveis
while True:
    print("\nOpções:")
    print("1. Criptografar")
    print("2. Descriptografar")
    print("3. Criar Chave Aleatória")
    print("4. Verificar Chave")
    print("5. Sair")
    opcao = input("Escolha uma opção: ")

# Opção 1, criptografia
if opcao == '1':
    # Aqui ele verifica se a chave inserida existe, caso não existe ele
informa ao usuario.
    chave_nome = input("Nome da chave ou chave hexadecimal: ")

```

```

chave = criptografia.buscar_chave(chave_nome)

# Caso a chave seja existente, ele inicia o processo de criptografia
usando exclusive OR, transformando o texto inserido em um código
hexadecimal de acordo com a chave.

if chave:
    mensagem = input("Insira a frase que deseja criptografar: ")
    if mensagem:
        mensagem_bytes = mensagem.encode('utf-8')
        mensagem_cifrada = criptografia.criptografar(mensagem_bytes,
chave)

        print("Frase criptografada:", mensagem_cifrada.hex())
    else:
        print("Nenhuma frase inserida. Operação de criptografia
cancelada.")
else:
    print("Chave não encontrada ou inválida.")

if opcao == '2':
    chave_nome = input("Nome da chave ou chave hexadecimal: ")
    chave = criptografia.buscar_chave(chave_nome)
    if chave:
        mensagem_cifrada_hex =
criptografia.input_mensagem_cifrada_hex()
        mensagem_cifrada = bytes.fromhex(mensagem_cifrada_hex)
        mensagem_decifrada =
criptografia.descriptografar(mensagem_cifrada, chave)
        mensagem_decifrada_str = mensagem_decifrada.decode('utf-8',
errors='ignore')
        print("Frase descriptografada:", mensagem_decifrada_str)
    else:
        print("Chave não encontrada ou inválida.")

```

#Opção 3, opção que dá ao usuário a opção de gerar uma chave 'string' e gerar uma chave

```
elif opcao == '3':  
    chave_nome = input("Nome da nova chave: ")  
    if chave_nome not in criptografia.chaves:  
        criptografia.criar_chave_aleatoria(chave_nome)  
        print("Chave aleatória gerada e associada a", chave_nome)  
    else:  
        print("Já existe uma chave com esse nome.")
```

Opção 4, essa opção busca no dicionário salvo a chave inserida, caso exista ela mostra a chave gerada que foi associada a inserida.

```
elif opcao == '4':  
    chave_nome = input("Digite o nome da chave para verificar: ")  
    if chave_nome in criptografia.chaves:  
        chave_bytes = criptografia.chaves[chave_nome]  
        print(f"Chave associada a {chave_nome}: {chave_bytes.hex()}")  
    else:  
        print("Chave não encontrada.")
```

Opção 5, fecha o programa.

```
elif opcao == '5':  
    break
```

REFERENCIAS

Acervo Lima. **Criptografia XOR por deslocamento de texto simples.**

Acervo Lima, 2022. Disponível em: <https://acervolima.com/criptografia-xor-por-deslocamento-de-texto-simples/>. Acesso em: 20 de outubro de 2023.

Amazon Web Services. **O que é criptografia? - Criptografia explicada**

- **AWS.** Disponível em: <https://aws.amazon.com/pt/whatis/cryptography/#:~:text=Criptografia%20%C3%A9%20a%20pr%C3%A1tica%20de>.

Gabriel54. **Codificação em Python para criptografia no modelo XOR simétrico.** Disponível em:

<https://www.clubedohardware.com.br/forums/topic/1496304-codifica%C3%A7%C3%A3o-em-python-para-criptografia-no-modelo-xor-sim%C3%A9trico/>. Acesso em: 21 out. 2023.

Valide Certificadora. **Tipos de criptografia: conheça os 10 mais usados e como funciona cada um.** Disponível em: <https://blog.validcertificadora.com.br/tipos-de-criptografia-conheca-os-10-mais-usados-e-como-funciona-cada-um/#:~:text=1.>>. Acesso em: 21 out. 2023.

XOR e a cifra de chave única. Khan Academy, 2023. Disponível em: <https://pt.khanacademy.org/computing/computer-science/cryptography/ciphers/a/xor-and-the-one-time-pad>. Acesso em: 20 de outubro de 2023.

XOR. Solitude Lab, 2019. Disponível em:

[https://solitudelab.wordpress.com/2019/08/04/xor/#:~:text=TL%3BDR-,XOR%20\(Exclusive%20OR\)%20%C3%A9%20um%20operador%20l%C3%B3gico%20booleano%20que%20recebe,mas%20n%C3%A3o%20ambas%2C%20s%C3%A3o%20true/](https://solitudelab.wordpress.com/2019/08/04/xor/#:~:text=TL%3BDR-,XOR%20(Exclusive%20OR)%20%C3%A9%20um%20operador%20l%C3%B3gico%20booleano%20que%20recebe,mas%20n%C3%A3o%20ambas%2C%20s%C3%A3o%20true/) . Acesso em: 20 de outubro de 2023.